

Contents

Task 1	2
A	2
B	2
C	2
Task 2	3
Task 3	3
Task 4	4
Command 1 – scp	4
Command 2 – rsync	4
Task 5	6
Disabling Connection as Root User	6
Admins Group and 2 Users	6
Info about AllowGroups, AllowUsers	8
Task 6	9
Task 7	9
Task 8	10
Task 9	11
Prohibit Log in with Password in Server 1	11
Generating Public and Private Keys	11
Sending Public Key to Server 1	12
Checking .ssh Directories in Both Servers	13
Connection	13
Verifying Connection with Password is Disabled	14

Task 1

A

This command (`sed -i s/^SELINUX=.*/SELINUX=permissive/ /etc/selinux/config`) turns SELinux off permanently. It means even after rebooting, SELinux is off. So, SELinux will not block anything.

I will not prefer this way in this task questions. Instead, I will block SELinux temporarily which I will explain below.

B

This command (`setenforce 0`) turns SELinux off temporarily. It means after rebooting, SELinux will start to work properly.

I would prefer this way for this task questions.

```
[root@rhel-9 ~ #] setenforce 0
```

C

We also need to turn firewall off. Otherwise, it can block the requests.

```
[root@rhel-9 ~ #] systemctl stop firewalld
[root@rhel-9 ~ #] systemctl status firewalld
o firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; preset: enabled)
   Active: inactive (dead) since Mon 2026-04-27 23:54:48 +04; 9s ago
     Duration: 10min 49.827s
    Docs: man:firewalld(1)
   Process: 1971 ExecStart=/usr/sbin/firewalld --nofork --nopid $FIREWALLD_ARGS (code=exited, status=0/SUCCESS)
   Process: 1973 ExecStartPost=/usr/bin/firewall-cmd --state (code=exited, status=0/SUCCESS)
   Main PID: 1971 (code=exited, status=0/SUCCESS)
      CPU: 425ms

Apr 27 23:43:58 rhel-9.7 systemd[1]: Starting firewalld - dynamic firewall daemon...
Apr 27 23:43:58 rhel-9.7 systemd[1]: Started firewalld - dynamic firewall daemon.
Apr 27 23:54:48 rhel-9.7 systemd[1]: Stopping firewalld - dynamic firewall daemon...
Apr 27 23:54:48 rhel-9.7 systemd[1]: firewalld.service: Deactivated successfully.
Apr 27 23:54:48 rhel-9.7 systemd[1]: Stopped firewalld - dynamic firewall daemon.
```

As you can see, I also prefer to turn it off temporarily. After rebooting, it will turn on.

Task 2

I have two Linux servers used now. One is RHEL 9.7 and the other is RHEL 10.1. I connected both via SSH.

Server 1 --- RHEL 9.7

Server 2 --- RHEL 10.1

Let's start.

I have turned SELinux off in Server 1. (in Task 1)

We can change SSH port:

```
[root@rhel-9 ~ #] cd /etc/ssh/  
[root@rhel-9 /etc/ssh #] vim sshd_config
```

Here, we find this line and change port number there:

```
Port 2222
```

 (Then save the file and quit)

Then we need to restart `sshd` service:

```
[root@rhel-9 /etc/ssh #] systemctl restart sshd
```

Task 3

BTW, I have allowed logging in as a root via SSH. `PermitRootLogin yes`

Now let's connect to Server 1 from Server 2.

```
[root@rhel-10 ~]# ssh -p 2222 root@192.168.88.160
root@192.168.88.160's password:
Activate the web console with: systemctl enable --now cockpit.socket

Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Tue Apr 28 00:12:07 2026 from 192.168.88.145
[root@rhel-9 ~ #]
```

I mentioned port as -p 2222 and write IP of Server 1 there to connect it successfully.

Task 4

We have two commands to copy files via SSH: **scp** and **rsync**.

Command 1 – scp

We write it in Server 2:

```
[root@rhel-10 ~]# scp -P 2222 /etc/hosts root@192.168.88.160:/tmp
root@192.168.88.160's password:
hosts
```

Let's check it in Server 1:

```
[root@rhel-9 ~ #] cd /tmp
[root@rhel-9 /tmp #] ll | grep "hosts"
-rw-r--r--. 1 root root 384 Apr 28 00:32 hosts
[root@rhel-9 /tmp #] cat hosts
# Loopback entries; do not change.
# For historical reasons, localhost precedes localhost.localdomain:
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
# See hosts(5) for proper format and other examples:
# 192.168.1.10 foo.example.org foo
# 192.168.1.13 bar.example.org bar
```

Command 2 – rsync

(First, let's remove the hosts file from /tmp in Server 1 to prevent confusion. So, we have copied it here with Command 1.

```
[root@rhel-9 /tmp #] rm -rf hosts
[root@rhel-9 /tmp #] ll | grep "hosts"
[root@rhel-9 /tmp #] )
```

Let's use the second command in Server 2:

(BTW in RHEL 10.1, there is no `rsync` package by default, so I have downloaded it)

```
[root@rhel-10 ~]# yum whatprovides rsync
Extra Packages for Enterprise Linux 9 - x86_64
Last metadata expiration check: 0:00:07 ago on Tue 28 Apr 2026 12:38:13 AM +04.
rsync-3.4.1-2.el10.x86_64 : A program for synchronizing files over a network
Repo      : mnt_BaseOS
Matched from:
Provide   : rsync = 3.4.1-2.el10

[root@rhel-10 ~]#
[root@rhel-10 ~]# yum install -y rsync-3.4.1-2.el10.x86_64
```

Now we can run:

```
[root@rhel-10 ~]# rsync -av -e "ssh -p 2222" /etc/hosts root@192.168.88.160:/tmp/
root@192.168.88.160's password:
sending incremental file list
hosts

sent 481 bytes  received 35 bytes  147.43 bytes/sec
total size is 384  speedup is 0.74
```

Here, `-a` means archive mode which keeps the permissions, too. Moreover, `-v` is verbose which shows what happens while copying. `-e "ssh -p 2222"` means to use Port 2222 for SSH. Then we write the source and then the destination.

Let's check it in Server 1:

```
[root@rhel-9 ~ #] cd /tmp
[root@rhel-9 /tmp #] ll | grep "hosts"
-rw-r--r--. 1 root root 384 Nov 29 2023 hosts
[root@rhel-9 /tmp #] cat hosts
# Loopback entries; do not change.
# For historical reasons, localhost precedes localhost.localdomain:
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1        localhost localhost.localdomain localhost6 localhost6.localdomain6
# See hosts(5) for proper format and other examples:
# 192.168.1.10 foo.example.org foo
# 192.168.1.13 bar.example.org bar
```

Task 5

Disabling Connection as Root User

We need to go to this file:

```
[root@rhel-9 ~ #] vim /etc/ssh/sshd_config
```

Then we find line with `PermitRootLogin` and write `no` here:

```
PermitRootLogin no
```

Then we go to `/etc/ssh/sshd\_config.d/` directory and find the file there:

```
[root@rhel-9 ~ #] cd /etc/ssh/sshd_config.d
[root@rhel-9 /etc/ssh/sshd_config.d #] ls
01-permitrootlogin.conf 50-redhat.conf
[root@rhel-9 /etc/ssh/sshd_config.d #] vim 01-permitrootlogin.conf
```

Then open this file and make the same change here, too:

```
# This file has been generated by the Anaconda Installer.
# Allow root to log in using ssh. Remove this file to opt-out.
PermitRootLogin no
```

Then we just need to restart sshd service:

```
[root@rhel-9 ~ #] systemctl restart sshd
```

We can check it by trying to connect to the server via SSH:

```
2026-04-28 01:52:59 /home/mobaxterm ssh root@192.168.88.160 -p 2222
root@192.168.88.160's password:
root@192.168.88.160's password:
root@192.168.88.160: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
```

(I have entered the password correctly)

Admins Group and 2 Users

As you can see from the image below, I have created `admins` group and 2 users. Then add these users to admins group.

```

[root@rhel-9 ~ #] groupadd admins
[root@rhel-9 ~ #] tail -n1 /etc/group
admins:x:1010:
[root@rhel-9 ~ #] useradd Ehmed_ADMIN && passwd Ehmed_ADMIN
Changing password for user Ehmed_ADMIN.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@rhel-9 ~ #] useradd Nihad_ADMIN && passwd Nihad_ADMIN
Changing password for user Nihad_ADMIN.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@rhel-9 ~ #] usermod -aG admins Ehmed_ADMIN
[root@rhel-9 ~ #] usermod -aG admins Nihad_ADMIN
[root@rhel-9 ~ #] id Ehmed_ADMIN
uid=1008(Ehmed_ADMIN) gid=1011(Ehmed_ADMIN) groups=1011(Ehmed_ADMIN),1010(admins)
[root@rhel-9 ~ #] id Nihad_ADMIN
uid=1009(Nihad_ADMIN) gid=1012(Nihad_ADMIN) groups=1012(Nihad_ADMIN),1010(admins)
[root@rhel-9 ~ #]

```

Then we open the file: `[root@rhel-9 ~ #] vim /etc/ssh/sshd_config`,

and add this line there: `AllowGroups admins`. Then restart the service:

```

[root@rhel-9 ~ #] systemctl restart sshd

```

Let's check it by trying to connect as other users (existing in Server 1):

```

2026-04-28 02:05:26 /home/mobaxterm ssh mhd1@192.168.88.160 -p 2222
mhd1@192.168.88.160's password:
mhd1@192.168.88.160's password:
mhd1@192.168.88.160's password:

2026-04-28 02:05:53 /home/mobaxterm ssh mhd1@192.168.88.160 -p 2222
mhd1@192.168.88.160's password:
mhd1@192.168.88.160's password:

2026-04-28 02:06:13 /home/mobaxterm ssh said@192.168.88.160 -p 2222
said@192.168.88.160's password:
said@192.168.88.160's password:
said@192.168.88.160's password:

```

No other user is allowed to connect via SSH.

However, when we try to connect as members of admins group:

```

2026-04-28 02:06.25 /home/mobaxterm ssh Nihad_ADMIN@192.168.88.160 -p 2222
Nihad_ADMIN@192.168.88.160's password:
Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
/usr/bin/xauth: file /home/Nihad_ADMIN/.Xauthority does not exist
[Nihad_ADMIN@rhel-9 ~]$

```

```

2026-04-28 02:08.53 /home/mobaxterm ssh Ehmed_ADMIN@192.168.88.160 -p 2222
Ehmed_ADMIN@192.168.88.160's password:
Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last failed login: Tue Apr 28 02:08:50 +04 2026 from 192.168.88.1 on ssh:notty
There was 1 failed login attempt since the last successful login.
Last login: Tue Apr 28 02:05:20 2026 from 192.168.88.1
[Ehmed_ADMIN@rhel-9 ~]$

```

Info about AllowGroups, AllowUsers

- **AllowGroups** – Only the members of the groups mentioned here can connect to the server via SSH.
- **AllowUsers** - Only the users mentioned here can connect to the server via SSH.

Task 6

```
[root@rhel-10 ~]# ssh -p 2222 Ehmed_ADMIN@192.168.88.160
Ehmed_ADMIN@192.168.88.160's password:
Register this system with Red Hat Insights: rhc connect
```

Example:

```
# rhc connect --activation-key <key> --organization <org>
```

The rhc client and Red Hat Insights will enable analytics and additional management capabilities on your system.

View your **connected** systems at <https://console.redhat.com/insights>

You can learn more about how to register your system using rhc at <https://red.ht/registration>

Last login: Tue Apr 28 02:08:59 2026 from 192.168.88.1

```
[Ehmed_ADMIN@rhel-9 ~ $]
```

```
[Ehmed_ADMIN@rhel-9 ~ $] who am i
```

```
Ehmed_ADMIN pts/1          2026-04-28 02:13 (192.168.88.145)
```

Task 7

```
[root@rhel-9 ~ #] ssh -t root@192.168.88.145 "lsblk" #Example Command
The authenticity of host '192.168.88.145 (192.168.88.145)' can't be established.
ED25519 key fingerprint is SHA256:gRfRy3Z5Tsp/LM0cRl5lB2iW9B04cGUjSFx20IRwnys.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.88.145' (ED25519) to the list of known hosts.
root@192.168.88.145's password:
NAME                MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
sda                  8:0    0   5G  0 disk
└─sda1               8:1    0   5G  0 part [SWAP]
sdb                  8:16   0    7G  0 disk
├─sdb1               8:17   0    5G  0 part
│   └─vg01-lv01      253:1   0    5G  0 lvm   /my-directory
└─sdb2               8:18   0    2G  0 part
    └─vg01-lv01      253:1   0    5G  0 lvm   /my-directory
sr0                  11:0    1  9.5G  0 rom  /mnt
nvme0n1              259:0   0   10G  0 disk
├─nvme0n1p1          259:1   0  600M  0 part /boot/efi
├─nvme0n1p2          259:2   0    1G  0 part /boot
├─nvme0n1p3          259:3   0   8.4G  0 part
└─rhel-root          253:0   0   8.4G  0 lvm   /
Connection to 192.168.88.145 closed.
[root@rhel-9 ~ #]
```

Task 8

We need to go this file in Server 2:

```
[root@rhel-10 ~]# vim /etc/ssh/sshd_config
```

Find this line, uncomment it and specify the given IP:

```
ListenAddress 127.0.0.1
```

Then we should restart the sshd service:

```
[root@rhel-10 ~]# systemctl restart sshd
```

Then we can check with netstat:

```
[root@rhel-10 ~]# netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:22            0.0.0.0:*               LISTEN      2490/sshd: /usr/sbi
udp        0      0 127.0.0.1:323          0.0.0.0:*               1140/chronyd
udp6       0      0 :::323                 :::*                     1140/chronyd
```

In that case if we try to connect to Server 2, it will fail:

```
2026-04-28 02:25.49 /home/mobaxterm ssh root@192.168.88.145
ssh: connect to host 192.168.88.145 port 22: Connection refused
```

Because this IP is loopback ip. We can only connect from inside the server.

To convert the changes back, we need to just remove this loopback ip from the config file of ssh and comment it again:

```
[root@rhel-10 ~]# vim /etc/ssh/sshd_config
```

```
#ListenAddress 0.0.0.0 ('0.0.0.0' means all IP are acceptable)
```

Then we restart the sshd service:

```
[root@rhel-10 ~]# systemctl restart sshd
```

Task 9

Prohibit Log in with Password in Server 1

To prohibit log in with password in Server 1, we need to go to ssh config file:

```
[root@rhel-9 ~ #] vim /etc/ssh/sshd_config
```

To disable the connection with password, we need to change it to `no` (completely disable connection with password):

```
# To disable tunneled clear text passwords, change to no here!  
PasswordAuthentication no
```

To check it:

```
2026-04-28 02:40.56 /home/mobaxterm ssh Ehmed_ADMIN@192.168.88.160 -p 2222  
Ehmed_ADMIN@192.168.88.160: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

To enable the connection with SSH Key (public and private stuff), we need to change it to `yes`:

```
PubkeyAuthentication yes
```

Generating Public and Private Keys

We use `ssh-keygen` method to generate the keys. In Server 2, let's run this command:

```
[root@rhel-10 ~]# ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:c5FtBLN6Wak2Zg0d0bw/GIu+tlnqsF02rbU95i0FwLg root@rhel-10.1
The key's randomart image is:
+--[ED25519 256]--+
|      B=++      |
|    + o*B       |
|      o*oo      |
|    .+=o        |
|    Soo* .      |
|    E+o.+ .     |
|    .. = oo     |
|    +o* +.o+    |
|    .oBo. .+oo  |
+-----[SHA256]-----+
[root@rhel-10 ~]#
```

Sending Public Key to Server 1

Let's send the generated public key to Server 1 (from Server 2). We need to use `ssh-copy-id`. However, to be able to send this to Server 1, we need to enable the connection with password temporarily, so we change this line to `yes` temporarily:

PasswordAuthentication yes, then restart sshd:

```
[root@rhel-9 ~ #] systemctl restart sshd
```

Now, let's try to send the public key from Server 2 to Server 1:

```
[root@rhel-10 ~]# ssh-copy-id -p 2222 Ehmed_ADMIN@192.168.88.160
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
Ehmed_ADMIN@192.168.88.160's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -p 2222 'Ehmed_ADMIN@192.168.88.160'"
and check to make sure that only the key(s) you wanted were added.

[root@rhel-10 ~]#
```

Now, we can disable the connection with password again in Server 1:

PasswordAuthentication no

Then we restart sshd service:

```
[root@rhel-9 ~ #] systemctl restart sshd
```

Checking .ssh Directories in Both Servers

In Server 2:

```
[root@rhel-10 ~]# cd .ssh/ && ll
total 16
-rw-----. 1 root root 411 Apr 28 02:44 id_ed25519
-rw-r--r--. 1 root root 96 Apr 28 02:44 id_ed25519.pub
-rw-----. 1 root root 861 Apr 28 00:11 known_hosts
-rw-r--r--. 1 root root 103 Apr 28 00:11 known_hosts.old
```

We have used `ssh-keygen` command, so the public and private keys are here.

In Server 1:

```
[Ehmed_ADMIN@rhel-9 ~ $] cd .ssh && ll
total 4
-rw-----. 1 Ehmed_ADMIN Ehmed_ADMIN 96 Apr 28 02:49 authorized_keys
[Ehmed_ADMIN@rhel-9 ~/.ssh $] cat authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIEQ3y2fr+pxsC0Q6PJ2B5ZXJu2XC55vJ52n09yfp7/jv root@rhel-10.1
[Ehmed_ADMIN@rhel-9 ~/.ssh $]
```

In `authorized_keys` file, there is a public key which is sent from Server 2 using `ssh-copy-id` command. (BTW, I switched to `Ehmed_ADMIN` user, because I have set the SSH connection for this user.)

Connection

Let's try to connect to Server 1 from Server 2:

```
[root@rhel-10 .ssh]# ssh -p 2222 Ehmed_ADMIN@192.168.88.160
Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Tue Apr 28 02:53:37 2026
[Ehmed_ADMIN@rhel-9 ~ $]
```

(I did not save the password for Server 1, beforehand. Moreover, I also blocked the connection with password in Server 1. So, be sure that this connection is with ssh keys)

Verifying Connection with Password is Disabled

We should do it in Server 1:

```
PasswordAuthentication no
```

Let's verify it. I have tried to connect to Server 1 from cmd in Windows. Generated SSH keys are not available for cmd, because MobaxTerm saves them in another place which is not known by cmd.

So, there is only one way for cmd: to use connection with password. However, I have blocked it, thus it should not work:

```
C:\Users\ravan>ssh -p 2222 Ehmed_ADMIN@192.168.88.160
Ehmed_ADMIN@192.168.88.160: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
C:\Users\ravan>
```